

MASTER STRATEGY & PRODUCT SPECIFICATION

100% Client-Side, Zero-Trust Multilingual Data Mapping Engine

Product Purpose: This document consolidates all previous architectural sessions into a single, comprehensive specification for the product team. It describes an intelligent data mapping workflow designed to ingest messy, unformatted, and multi-language Excel spreadsheets. To ensure data privacy, the entire pipeline executes locally on the user's browser CPU via WebAssembly (WASM)—without third-party tools, GPUs, or cloud services.

1. STRATEGIC DESIGN & GUARDRAILS

- **Strict Data Isolation:** Data never leaves the client device. Zero server calls, zero network uploads. Matches 100% air-gapped corporate data compliance.
- **Hardware Independence:** Operates entirely on standard customer browser threads. No expensive GPU cluster dependencies.
- **High-Velocity Target:** Total execution time remains under 200ms following initial asset model load.
- **Human-in-the-Loop Validation:** Algorithmic scores are presented to users via clear confidence groupings to avoid false matches.

2. END-TO-END PROCESSING ARCHITECTURE

The processing pipeline splits into 5 automated stages to protect processing threads from unnecessary workload overhead:

Workflow Stage	Underlying Engine	Functional Execution Responsibility
Stage 1: Fast Ingestion	SheetJS (xlsx) inside Web Worker	Parses local binary arrays into an in-memory 2D JavaScript array. Data remains entirely local.
Stage 2: Contextual Scoring	Deterministic Signal Evaluator (CSA)	Scans the first 20 lines to dynamically calculate and isolate the file's primary header row.
Stage 3: Confidence UI	Color-Coded Status Framework	Displays mapping match predictions to users for confirmation or rapid manual dropdown fixes.
Stage 4: Cascading Cascade	3-Layer Progressive Mapping Engine	Executes a hierarchical matching chain (Exact → Lexical → Vector) to maximize processing speed.
Stage 5: Data Extraction	Single-Pass High-Speed Mapping Loop	Transforms the remaining rows into clean JSON key-value datasets matching the target schema.

3. STAGE 2 DEEP-DIVE: CONTEXTUAL SCORING ALGORITHM (CSA)

Instead of assuming row 0 is the header, the CSA evaluates rows using a weighted multi-signal logic loop:

Signal Component	Weight Metric	Target Detection Behavior & Logic
String Ratio	0.35	Evaluates row text density. Header rows contain mostly text strings; data blocks mix types.
Non-Empty Density	0.20	Measures cell completeness. Headers span completely across without structural cell holes.
Unique Value Ratio	0.20	Evaluates cell repetitions. Header entries rarely duplicate labels within the identical row.
Row Position	0.15	Applies structural probability decay. Rows 0–3 receive higher scoring weight.
Short Cell Length	0.10	Evaluates character length. True header text fields use short labels; data content is long.

Core Evaluation Function:

```
function scoreRow(row, rowIndex) {
  const nonEmpty = row.filter(c => c !== '');
  const strRatio = nonEmpty.filter(c => isNaN(c)).length / (nonEmpty.length || 1);
  const density = nonEmpty.length / (row.length || 1);
  const uniqueRatio = new Set(row).size / (row.length || 1);
  const posScore = Math.max(0, 1 - rowIndex / 10);
  const avgLen = nonEmpty.reduce((a, c) => a + String(c).length, 0) / (nonEmpty.length || 1);
  const lenScore = avgLen < 30 ? 1 : 0.3;
  return 0.35 * strRatio + 0.20 * density + 0.20 * uniqueRatio + 0.15 * posScore + 0.10 * lenScore;
}
```

4. STAGE 3 DEEP-DIVE: PRODUCT CONFIDENCE UI WORKFLOW

Calculated composite columns map directly to three designated interaction zones to streamline verification:

Confidence Zone	Score Boundary	Product UI Design & Human-in-the-Loop Action
 High Confidence	≥ 80% Score Match	Auto-selects match with green badge. User can bypass review and single-click "Accept All".
 Medium Confidence	55% – 79% Score Match	Pre-selects best match with yellow caution icon. Prompts explicit dropdown confirmation.
 Low Confidence	< 55% Score Match	Left blank with a red warning badge. Forces manual column selection via dropdown list.

5. STAGE 4 DEEP-DIVE: 3-LAYER SHORT-CIRCUIT MAPPING CASCADE

To reduce CPU load, the system uses a progressive short-circuit evaluation pipeline. Heavy AI modeling is triggered only if simpler lexical checks fail.

Cascading Target Layer	CPU Cost	Algorithmic Execution & Purpose
Layer 1: Exact + Alias Map	~0 ms	Applies uniform text normalization (lowercase, space-to-underscore) and flat dictionary lookups.
Layer 2: Lexical Recovery	1 – 5 ms	Runs BM25 token relevance for out-of-order words and character trigrams for typos.
Layer 3: Vector Semantic	10 – 25 ms	Executes local ONNX phrase embeddings and cosine similarity checks for unseen translations.

A. Layer 1 Execution Parameters (Deterministic Lookup)

- **Text Cleansing Sequence:** `toLowerCase()` → `trim()` → Strip special punctuation → Convert whitespace blocks to uniform underscores (`_`).
- **Alias Scale:** For 25 specific target keys across 5 global languages with 5-10 structural phrasings, developers manage a flat JSON dictionary of ~600–1,500 rows.

Target Key Mapping	English Layout Aliases	International Multilingual Static Lookups
customer_name	client_name, buyer_name, full_name	வாடிக்கையாளர்_பெயர் (Tamil), ग्राहक_नाम (Hindi), Kundennamen (German), اسم العميل (Arabic)
date_of_birth	birth_date, born_on, birthday	பிறந்த_தேதி (Tamil), जन्म_तिथि (Hindi), تاريخ الميلاد (Arabic), Geburtsdatum (German)

B. Layer 2 Execution Parameters (Lexical Recovery)

- **BM25 Pass:** Scores word rarity and token overlap to match reordered phrases (e.g., maps `"name of the customer"` to `customer_name`).
- **Trigram Jaccard Pass:** Breaks text into 3-character blocks to calculate structural overlap, repairing spelling errors (e.g., maps `"custmer_name"` to `customer_name`).

C. Layer 3 Execution Parameters (Vector Semantic Fallback)

- **Pre-Embedding Strategy:** Embeddings for the 25 target keys are computed **once** at application startup.
- **Runtime Overhead:** When an unknown column header arrives, the system runs exactly 1 vector conversion for that input string, followed by 25 cheap float-array cosine similarity operations.

Raw Incoming Header String	Layer 1 / Layer 2 Output	Layer 3 Strategy Action	Resolved Mapping Result
"né le" (French Input)	Null / Null	Vector Cosine Pass	date_of_birth (Score: 0.89)
"when were you born"	Null / Null	Vector Cosine Pass	date_of_birth (Score: 0.91)
"account holder identity"	Null / Fail (Score: 0.41)	Vector Cosine Pass	customer_name (Score: 0.78)
"system_code_xyz1"	Null / Null	Score Under Threshold	Null → Trigger Manual UI Dropdown

6. STRATEGIC TECHNOLOGY STACK EVALUATION

A. Layer 2 Alternatives (Lexical Processing Engine)

Chosen Library: `wink-bm25` (50KB size, pure JS, zero native binaries, standard engine framework).

Evaluated Option	Package Size	Product Team Evaluation & Rejection Reason
<code>lunr.js</code>	~30 KB	Uses legacy TF-IDF logic instead of modern, higher-precision BM25 scoring.
<code>minisearch</code>	~25 KB	Optimized explicitly for long document scanning rather than short phrase/label matches.
<code>fuse.js</code>	~24 KB	Limited to basic string character edits. Lacks conceptual keyword index structures.
<code>flexsearch</code>	~6 KB	Highly focused on text matching velocity at the explicit expense of precision ranking accuracy.

B. Layer 3 Alternatives (Embedding Model Selection)

Chosen Model: `paraphrase-multilingual-MiniLM-L12-v2` (118MB size, 50+ languages, optimized local CPU extraction).

Model Candidate	Disk Size	Language Support	Product Suitability & Rationale
<code>all-MiniLM-L6-v2</code>	22 MB	English Only	Rejected. Fails on global/multilingual headers.
<code>paraphrase-multilingual-MiniLM-L12-v2</code>	118 MB	50+ Languages	SELECTED. Ideal balance between file size, language support, and CPU speed.
<code>paraphrase-multilingual-mpnet-base-v2</code>	420 MB	50+ Languages	Rejected. Too large; slows down browser loading and spikes memory.
<code>LaBSE</code>	470 MB	109 Languages	Rejected. Overkill for short header strings; degrades client performance.

C. Execution Runtimes (Inference Engine)

Chosen Engine: `@xenova/transformers` with **ONNX Runtime Web** (Pure client-side CPU processing).

Runtime Platform	CPU Support	Browser Support	Product Risk Evaluation
TensorFlow.js	Yes (Slow)	All Browsers	Adds a heavy 5MB runtime size library footprint.
ONNX Runtime (Node)	Yes	None (Server Bound)	Restricted to native servers; fails local browser operations.
llama.cpp WASM	Yes	Limited Availability	Requires excessive technical orchestration and initialization overhead.

7. TECHNICAL IMPLEMENTATION GUIDELINES

- **Mandatory Web Worker Isolation:** Layer 3 model loading and vector calculations must execute inside a background Web Worker thread. This isolates processing-heavy operations and prevents the browser UI from freezing during the file upload workflow.
- **Automatic IndexedDB Client Caching:** The 118MB multilingual model file downloads only on the user's very first visit. It is then securely cached in the browser's local IndexedDB storage. On subsequent visits, the model loads directly from the local disk in under 500ms.
- **Row Ingestion Bypass:** The system extracts only the first 20 rows of a file to calculate data types, patterns, and mapping relationships. Once the user approves the configuration, the app switches to a high-speed data stream pass to process the rest of the sheet.